UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Lab Report – 05

COURSE CODE: CSE 402

COURSE TITLE: Operating System Lab

SUBMITTED BY:

NAME : Swagoto Utsab Singha Roy

STUDENT ID : 23101124

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C–1

DATE : 12 – 10 – 2026

SUBMITTED TO:

ATIA RAHMAN ORTHI
LECTURER
DEPARTMENT OF CSE, UAP

1. Problem Statement

Implement the **Priority Scheduling** Algorithm in two variants: **(A) Priority Scheduling with Time Quantum** and **(B) Non-Preemptive and Preemptive Priority Scheduling**, for the following set of processes (lower priority number = higher priority):

Process	Arrival Time (AT)	Burst Time (BT)	Priority (PR)
P1	3	4	2
P2	0	4	0
P3	2	3	1
P4	1	6	1
P5	4	2	3

For all scheduling variants, calculate:

- Completion Time (CT)
- Turnaround Time (TAT = CT − AT)
- Waiting Time (WT = TAT − BT)
- Average Turnaround Time (Avg TAT)
- Average Waiting Time (Avg WT)

Part A – Priority Scheduling with Time Quantum: Processes are grouped into priority-level queues. The scheduler always picks from the highest-priority non-empty queue. Within the same priority level, Round Robin is applied using the given Time Quantum (TQ). The program runs for TQ = 1, 2, and 3.

Part B – Non-Preemptive Priority Scheduling: Once a process is selected it runs to full completion without interruption.

Part B – Preemptive Priority Scheduling: The scheduler re-evaluates at every clock tick; a higher-priority arrival immediately preempts the running process.

2. Source Code

Part A: Priority Scheduling with Time Quantum

```python
from collections import deque

process = ["P1", "P2", "P3", "P4", "P5"]
at      = [3, 0, 2, 1, 4]
bt      = [4, 4, 3, 6, 2]
pr      = [2, 0, 1, 1, 3]

N = len(process)


def table(title, ct: {__getitem__}):  1 usage  new *
    tat = [ct[i] - at[i] for i in range(N)]
    wt  = [tat[i] - bt[i] for i in range(N)]

    print(f"\n{title}")
    print(f"{'Process':<10}{'AT':<6}{'BT':<6}{'PR':<6}{'CT':<6}{'TAT':<6}{'WT':<6}")
    for i in range(N):
        print(f"{process[i]:<10}{at[i]:<6}{bt[i]:<6}{pr[i]:<6}{ct[i]:<6}{tat[i]:<6}{wt[i]:<6}")
    print(f"Avg TAT = {sum(tat)/N:.2f}")
    print(f"Avg WT  = {sum(wt)/N:.2f}")


def priority(tq):  3+ usages  new *
    rem    = bt.copy()
    ct     = [0] * N
    q      = {p: deque() for p in sorted(set(pr))}
    added  = set()
    time   = 0
    done   = 0

    while done < N:
        for i in range(N):
            if i not in added and at[i] <= time:
                q[pr[i]].append(i)
                added.add(i)

        if not any(q.values()):
            time += 1
            continue

        p = min(k for k in q if q[k])
        i = q[p].popleft()

        run = min(tq, rem[i])
        for _ in range(run):
            rem[i] -= 1
            time += 1
            for j in range(N):
                if j not in added and at[j] <= time:
                    q[pr[j]].append(j)
                    added.add(j)

        if rem[i] == 0:
            ct[i] = time
            done += 1
        else:
            q[p].append(i)

    table( title: f"Priority Scheduling (Lower # = Higher Priority) | TQ = {tq}", ct)


priority(1)
priority(2)
priority(3)
```

Listing 1: Priority Scheduling with Time Quantum (`Priority_with_tq.py`)

Part B: Non-Preemptive and Preemptive Priority Scheduling

```
process = ["P1", "P2", "P3", "P4", "P5"]
at      = [3, 0, 2, 1, 4]
bt      = [4, 4, 3, 6, 2]
pr      = [2, 0, 1, 1, 3]

n = len(process)


def print_table(title, ct_list: {__getitem__}): 2+ usages  new *
    total_tat = 0
    total_wt  = 0
    print(f"\n{title}")
    print("Process\tAT\tBT\tPR\tCT\tTAT\tWT")
    for i in range(n):
        tat = ct_list[i] - at[i]
        wt  = tat - bt[i]
        total_tat += tat
        total_wt  += wt
        print(f"{process[i]}\t{at[i]}\t{bt[i]}\t{pr[i]}\t{ct_list[i]}\t{tat}\t{wt}")
    print(f"Avg TAT = {total_tat / n:.2f}")
    print(f"Avg WT  = {total_wt  / n:.2f}")

ct   = [0] * n
done = [False] * n
time = 0
completed = 0

while completed < n:
    best = -1
    for i in range(n):
        if at[i] <= time and not done[i]:
            if best == -1 or pr[i] < pr[best] or (pr[i] == pr[best] and at[i] < at[best]):
                best = i

    if best == -1:
        time += 1
        continue
    time += bt[best]
    ct[best] = time
    done[best] = True
    completed += 1

print_table( title: "Non-Preemptive Priority Scheduling", ct)

ct        = [0] * n
remaining = bt.copy()
time      = 0
completed = 0

while completed < n:
    best = -1
    for i in range(n):
        if at[i] <= time and remaining[i] > 0:
            if best == -1 or pr[i] < pr[best] or (pr[i] == pr[best] and at[i] < at[best]):
                best = i

    if best == -1:
        time += 1
        continue

    remaining[best] -= 1
    time += 1

    if remaining[best] == 0:
        ct[best] = time
        completed += 1

print_table( title: "Preemptive Priority Scheduling", ct)
```

Listing 2: Non-Preemptive and Preemptive Priority Scheduling (`Priority_Non_Preemptive_Preemptive.py`)

3. Output

A – Priority Scheduling with Time Quantum (TQ = 1, 2, 3)

```
Priority Scheduling (Lower # = Higher Priority) | TQ = 1
Process   AT    BT    PR    CT    TAT   WT
P1        3     4     2     17    14    10
P2        0     4     0     4     4     0
P3        2     3     1     10    8     5
P4        1     6     1     13    12    6
P5        4     2     3     19    15    13
Avg TAT = 10.60
Avg WT  = 6.80

Priority Scheduling (Lower # = Higher Priority) | TQ = 2
Process   AT    BT    PR    CT    TAT   WT
P1        3     4     2     17    14    10
P2        0     4     0     4     4     0
P3        2     3     1     11    9     6
P4        1     6     1     13    12    6
P5        4     2     3     19    15    13
Avg TAT = 10.80
Avg WT  = 7.00

Priority Scheduling (Lower # = Higher Priority) | TQ = 3
Process   AT    BT    PR    CT    TAT   WT
P1        3     4     2     17    14    10
P2        0     4     0     4     4     0
P3        2     3     1     10    8     5
P4        1     6     1     13    12    6
P5        4     2     3     19    15    13
Avg TAT = 10.60
Avg WT  = 6.80
```

Output of `Priority_with_tq.py`

Summary – Average Results by Time Quantum:

Time Quantum	Avg TAT	Avg WT
TQ = 1	10.60	6.80
TQ = 2	10.80	7.00
TQ = 3	10.60	6.80

B – Non-Preemptive and Preemptive Priority Scheduling

```
Non-Preemptive Priority Scheduling
Process AT  BT  PR  CT  TAT WT
P1  3   4   2   17  14  10
P2  0   4   0   4   4   0
P3  2   3   1   13  11  8
P4  1   6   1   10  9   3
P5  4   2   3   19  15  13
Avg TAT = 10.60
Avg WT  = 6.80

Preemptive Priority Scheduling
Process AT  BT  PR  CT  TAT WT
P1  3   4   2   17  14  10
P2  0   4   0   4   4   0
P3  2   3   1   13  11  8
P4  1   6   1   10  9   3
P5  4   2   3   19  15  13
Avg TAT = 10.60
Avg WT  = 6.80
```

Output of `Priority_Non_Preemptive_Preemptive.py`

Non-Preemptive – Results:

P	AT	BT	PR	CT	TAT	WT
P1	3	4	2	17	14	10
P2	0	4	0	4	4	0
P3	2	3	1	13	11	8
P4	1	6	1	10	9	3
P5	4	2	3	19	15	13
				Avg	**10.60**	**6.80**

Preemptive – Results:

P	AT	BT	PR	CT	TAT	WT
P1	3	4	2	17	14	10
P2	0	4	0	4	4	0
P3	2	3	1	13	11	8
P4	1	6	1	10	9	3
P5	4	2	3	19	15	13
				Avg	**10.60**	**6.80**

4. Discussion

In this experiment, three variants of Priority Scheduling were implemented and compared using a fixed set of five processes (P1–P5) where a *lower priority number means higher priority.*

Part A – Priority Scheduling with Time Quantum (TQ): The scheduler maintains one queue per priority level and always drains the highest-priority non-empty queue first. Within the same priority level, processes share the CPU in Round-Robin order using the specified time quantum. P2 holds the highest priority (PR = 0) and arrives at time 0, so it runs first and completes at CT = 4. P3 and P4 share priority level 1 and alternate under Round Robin. P1 (PR = 2) and P5 (PR = 3) wait behind them.

A time quantum of TQ = 2 yields slightly higher averages (Avg TAT = 10.80, Avg WT = 7.00) compared to TQ = 1 and TQ = 3 (both 10.60 / 6.80). This happens because P3 (BT = 3) cannot fill a quantum of 2 evenly: it executes for 2 units, is re-queued, and executes for 1 more unit, introducing a small extra delay compared to when TQ = 1 (no fragmentation) or TQ = 3 (P3 finishes within a single quantum).

Part B – Non-Preemptive vs. Preemptive:

- **Non-Preemptive:** Once the highest-priority available process is selected, it runs to full completion (`time += bt[best]`). No newly arriving process can interrupt it.
- **Preemptive:** The scheduler rechecks every clock tick (`remaining[best] -= 1; time += 1`). A higher-priority arrival would immediately displace the current process.

For this dataset both algorithms produce *identical* output (Avg TAT = 10.60, Avg WT = 6.80 for both), because all five processes have arrived by time 4 – exactly when P2 (the highest-priority process) finishes. With no new arrivals after t = 4, the preemptive scheduler never finds a reason to interrupt; it makes exactly the same decisions as the non-preemptive one. This illustrates that the two variants diverge only when a *higher-priority* process arrives *while* a lower-priority one is running.

Starvation: In all variants, P5 (PR = 3, the lowest priority) is always scheduled last, finishing at CT = 19 with WT = 13. If higher-priority processes kept arriving indefinitely, P5 could be starved. The standard remedy is *aging*: gradually increasing a process's priority the longer it waits.

5. Conclusion

This lab demonstrated three forms of Priority Scheduling over the same process set. Part A showed that Priority Scheduling with Time Quantum combines strict priority ordering with Round-Robin fairness inside each level; the choice of time quantum can slightly affect average waiting time when burst times are not divisible by the quantum. Part B confirmed that Non-Preemptive and Preemptive Priority Scheduling produce identical schedules for this particular dataset, because all processes arrive before any post-P2 scheduling decision is needed. In general, Preemptive Priority Scheduling reacts faster to high-priority arrivals but incurs more context switches, while Non-Preemptive Priority Scheduling is simpler and context-switch-free at the cost of holding the CPU even when a more urgent job

arrives. Both variants require aging or a similar mechanism to prevent indefinite starvation of low-priority processes.

6. GitHub

Click here to view source code